MetaGuard

Metadata Analysis & Scrubbing Tool

Metadata Leakage as a Web Security Vulnerability

Nagham Mhesen (נגם מחיסן)

Azrieli College of Engineering – School of Software Engineering

Web Security Course

Table of Contents

1. Vulnerability Description & Theoretical Background *4*

1.1 What is Metadata Leakage? 4

1.2 How Metadata is Created 4

1.3 Where This Vulnerability Appears in Web Systems 4

1.4 What an Attacker Can Achieve 5

2. Working Environment *6*

2.1 Technology Stack 6

2.2 Key Features 6

2.3 Risk Scoring System 6

2.4 System Architecture 7

2.5 Demonstration Tools 7

3. Proof of Concept (POC) - Attack Demonstration *9*

3.1 POC Description 9

3.2 Attack Scenario - Exploitation Steps 9

3.3 Example Payloads & API Requests 9

Analyze a File (Expose Metadata & Risk Assessment) 9

Verify High-Risk Metadata (Without Modifying File) 10

Scrub Metadata & Obtain Download Link 10

Download the Scrubbed File 10

3.4 Step-by-Step User Guide 10

Installation 10

Using the Interactive Swagger UI 10

Downloading Scrubbed Files 11

Recommended Workflow 11

3.5 API Response Examples 11

/analyze Response Structure 11

/scrub Response Structure 11

/verify Response Structure 12

3.6 Attack Flow Diagram 12

4. Defense Mechanisms & Prevention *13*

4.1 Threat Model 13

4.2 Why Standard Defenses Fail 13

4.3 How to Prevent Metadata Leakage 13

4.4 Relevant Security Best Practices 14

4.5 Security Hardening Summary 14

5. Project Structure & Module Reference *15*

6. Design Decisions & Limitations ***16***

6.1 Design Tradeoffs **16**

In-Memory Rate Limiting vs. Redis 16

Stateless Architecture (No Database) 16

5-Minute TTL for Scrubbed Files 16

Rule-Based Risk Engine vs. Machine Learning 16

File-Based Temporary Storage vs. In-Memory Processing 16

6.2 Known Limitations **16**

7. References ***18***

Academic & Research Sources **18**

Official Documentation **18**

OWASP References **18**

Relevant Links **18**

1. Vulnerability Description & Theoretical Background

1.1 What is Metadata Leakage?

Metadata leakage, also known as Information Disclosure through Metadata, is a web security vulnerability in which sensitive information is exposed not through a file's visible content, but through metadata fields embedded within its internal structure. Metadata is literally 'data about data': technical and structural descriptors that are not part of what the user sees, yet are stored inside the file's binary or XML structure.

In web systems, files uploaded by users or distributed to the public frequently carry rich metadata that many platforms never examine or remove, resulting in a direct breach of Confidentiality and Privacy.

1.2 How Metadata is Created

Metadata is generated automatically and stored in different locations depending on file type:

- **Images (EXIF - Exchangeable Image File Format):** JPEG, PNG, and TIFF files can embed EXIF data that includes the capture date and time, camera manufacturer and model, editing software, and critically, GPS coordinates of the location where the photo was taken. Smartphones and digital cameras write this data automatically without user awareness.
- **PDF Documents:** PDF metadata fields include Title, Author, Subject, Creator, Producer, CreationDate, and ModDate. These fields can reveal email addresses, usernames, software versions (e.g., Adobe Acrobat X.Y), and internal system identifiers.
- **Office Documents (DOCX):** Microsoft Word files store Core Properties including title, author, subject, comments, last_modified_by, and revision. These fields can expose the author's name, the last editor's network username, and organizational information.

In all cases, the information is stored within the file's internal structure, binary or compressed XML, and is not displayed in the user interface, but can be trivially extracted using standard tools or libraries such as Pillow, PyPDF2, and python-docx.

1.3 Where This Vulnerability Appears in Web Systems

This vulnerability arises in several high-impact contexts relevant to web security:

- **File Upload Systems:** Websites that allow image, document, or resume uploads often store files exactly as received, without stripping metadata. Those files may then be displayed to other users or exposed via download.
- **Public File Sharing Portals:** Portals distributing documents, images, or forms can distribute files with metadata that identifies authors, locations, or internal systems.
- **HR & Enterprise Portals:** Resumes and recruitment documents uploaded as Word or PDF files frequently expose author email addresses, corporate network usernames, and operating system versions via metadata.

- **Social Media & Media Platforms:** Photos uploaded with EXIF data (including GPS) can expose a home address or workplace location if the server does not strip metadata before storage or display.

In all of these contexts, the server acts as an unintentional relay of sensitive information because it does not process the metadata layer.

1.4 What an Attacker Can Achieve

Exploiting metadata leakage enables a range of attacks:

- **OSINT (Open Source Intelligence):** GPS coordinates from images enable mapping of sensitive locations. Author names and email addresses enable building a detailed profile of a person or organization.
- **GPS Exposure / Physical Tracking:** A photo taken at home or at an office with GPS enabled exposes an exact address, enabling physical surveillance or targeted attacks.
- **Spear Phishing:** Email addresses or usernames found in document metadata are used to craft highly credible, targeted phishing messages.
- **Software Fingerprinting:** Creator/Producer fields and version strings expose exact versions of Adobe Acrobat, Microsoft Office, or the operating system, allowing an attacker to select a matching known CVE exploit.

All of these attack vectors are directly tied to the Confidentiality pillar of information security in web systems.

2. Working Environment

2.1 Technology Stack

MetaGuard is implemented in Python and uses the following technologies and libraries:

- **Python:** Primary programming language.
- **FastAPI:** Web framework for building the RESTful API. Defines endpoints /analyze, /verify, /scrub, /download/{filename}, and handles UploadFile objects and validation.
- **Uvicorn:** ASGI server for running the application (host 0.0.0.0, port 8000).
- **Pillow:** Extracts and removes EXIF metadata (including GPS) from images in the analyzer and scrubber modules.
- **PyPDF2:** Reads PDF metadata (Author, Creator, Producer, dates) and removes it when writing a clean output file.
- **python-docx:** Accesses DOCX Core Properties (author, last_modified_by, revision, etc.) for extraction and removal.
- **python-magic:** Detects file MIME types from actual file content (not just extension) for validation in the file_validation module.

2.2 Key Features

- **Metadata Extraction:** From images (JPEG, PNG, GIF, BMP, TIFF), PDFs, and DOCX files.
- **Risk Assessment:** Rule-based scoring engine that rates metadata sensitivity.
- **Metadata Scrubbing:** Removes metadata from all supported file types regardless of risk level.
- **Secure File Handling:** MIME validation, 10MB size limit, cryptographically secure random filenames, automatic cleanup.
- **SHA-256 Hashing:** Computes file hashes before and after scrubbing for integrity verification.

2.3 Risk Scoring System

The risk engine (risk_engine.py) assigns points based on the type of metadata detected:

Metadata Field	Risk Points	Risk Level
GPS Coordinates (EXIF)	**+40**	High
Author Email Address	**+30**	High/Med
Username / Last Modified By	**+20**	High/Med
Software Version (Creator/Producer)	**+15**	Low
Camera Model	**+5**	Low
Camera Make / Manufacturer	**+5**	Low

Risk Level Thresholds:

- **Low (0–29 points):** Minimal privacy risk. Basic technical information only.
- **Medium (30–69 points):** Moderate risk. May contain identifying information.
- **High (70–100 points):** Significant risk. Contains multiple sensitive fields.

2.4 System Architecture

MetaGuard follows a stateless, database-free architecture built around clear separation of concerns:

- **Stateless Design:** No user sessions or persistent state. Each request is processed independently. Files are stored in a temporary directory and deleted after processing.
- **No Database:** No long-term storage of files or metadata. All information is used only for analysis, risk assessment, and temporary download availability.
- **Temporary File Storage:** Files are stored in tempfile.gettempdir()/metaguard using cryptographically random names (secrets.token_urlsafe(32)) with 0o600 permissions. Source files are deleted immediately after processing; scrubbed files are available for download for 5 minutes only.
- **In-Memory Rate Limiting:** The rate_limiter module implements per-IP sliding window rate limiting (20 requests per 60 seconds) using a defaultdict of deques, no Redis or database required.
- **Structured Security Logging:** The security_logging module records security events as JSON (file_analyzed, file_scrubbed, file_downloaded, rate_limit_triggered) with masked IPs (last IPv4 octet replaced), file type, and risk level, never raw metadata values, file paths, coordinates, or file contents.

Scrubbed files are registered in a global dictionary with their creation timestamp. A TTL of 300 seconds (5 minutes) is enforced on every /download request and by periodic cleanup (_cleanup_expired_files). After TTL expiry, the file is deleted and download requests return HTTP 404.

2.5 Demonstration Tools

- **Swagger UI (http://localhost:8000/docs):** Interactive API documentation provided by FastAPI. Allows uploading files to /analyze, /verify, and /scrub directly in the browser, viewing JSON responses, and obtaining download_url for scrubbed files. Scrubbed files expire after 5 minutes, after expiry the URL returns 404 and the user must re-run /scrub.
- **curl:** Used for command-line API calls (see Proof of Concept section below).

MetaGuard 1.0.0 OAS 3.1

/openapi.json

Metadata analysis and scrubbing service for images, PDFs, and DOCX files

default

GET	/	Root
GET	/health	Health Check
POST	/analyze	Analyze File
POST	/verify	Verify File
POST	/scrub	Scrub File
GET	/download/{filename}	Download File

Schemas

Body_analyze_file_analyze_post	Expand all	object
Body_scrub_file_scrub_post	Expand all	object
Body_verify_file_verify_post	Expand all	object
HTTPValidationError	Expand all	object
ValidationError	Expand all	object

3. Proof of Concept (POC) - Attack Demonstration

3.1 POC Description

The Proof of Concept demonstrates how a naive web system that accepts files and redistributes them (or their metadata) without removal or risk assessment can expose sensitive information to any party who receives the file or calls the API.

MetaGuard serves as the detection and mitigation tool in this scenario: it extracts metadata, evaluates risk using the rule-based engine (risk_engine), and enables removal (scrubber) along with additional defensive controls (rate limiting, TTL, structured logging). The demonstration shows both what would have been exposed in an unprotected system, and how MetaGuard detects and mitigates the leakage.

3.2 Attack Scenario - Exploitation Steps

A realistic attack unfolds in three stages:

1. **Step 1: Attacker Acquires a Distributed File:** The attacker obtains a file distributed by a website, a photo published on a portal, a PDF downloaded from a form submission page, or a DOCX shared via a document-sharing system. The file was stored and served by the server without metadata processing, so it still contains its original EXIF, PDF Info, or Office Core Properties.
2. **Step 2: Metadata Extraction:** The attacker uses a standard tool such as exiftool, or writes a simple Python script using Pillow/PyPDF2/python-docx, to extract all metadata fields from the file. Alternatively, they call an API endpoint (such as /analyze in MetaGuard) that returns a structured metadata response. No authentication or special privileges are required.
3. **Step 3: Exploitation of Extracted Information:** GPS coordinates are used to map the exact location where the photo was taken (home, office, facility). Email addresses are used to craft spear phishing messages. Software version strings are used to identify matching CVE exploits. Usernames are used to enumerate internal directory structures or corporate accounts.

In MetaGuard, the attack phase is demonstrated by uploading a file with sensitive metadata to /analyze or /verify. The response shows exactly which fields are exposed and what the calculated risk level is, making the information disclosure concrete and measurable.

3.3 Example Payloads & API Requests

Analyze a File (Expose Metadata & Risk Assessment)

```
curl -X POST "http://localhost:8000/analyze" \
  -H "accept: application/json" \
  -H "Content-Type: multipart/form-data" \
  -F "file=@image_with_gps.jpg"
```

Verify High-Risk Metadata (Without Modifying File)

```
curl -X POST "http://localhost:8000/verify" \
  -H "accept: application/json" \
  -H "Content-Type: multipart/form-data" \
  -F "file=@document.pdf"
```

Scrub Metadata & Obtain Download Link

```
curl -X POST "http://localhost:8000/scrub" \
  -H "accept: application/json" \
  -H "Content-Type: multipart/form-data" \
  -F "file=@sensitive.docx"
```

Download the Scrubbed File

```
curl -X GET "http://localhost:8000/download/TOKEN_scrubbed.pdf" -o scrubbed.pdf
```

3.4 Step-by-Step User Guide

The following steps describe exactly how to use MetaGuard, from installation through downloading a clean file:

Installation

4. **Install Python dependencies:** pip install -r requirements.txt
5. **Install the libmagic system library:**

macOS: brew install libmagic

Ubuntu/Debian: sudo apt-get install libmagic1

Windows: Download from the libmagicwin64 repository on GitHub.

6. **Start the server:** python3 main.py or uvicorn main:app --reload

The API is now available at http://localhost:8000

Using the Interactive Swagger UI

7. **Step 1: Access Documentation:** Open your browser and navigate to http://localhost:8000/docs. You will see the full list of endpoints with descriptions.
8. **Step 2: Select an Operation:** Click any endpoint (/analyze, /scrub, or /verify) to expand its details. Read the description and parameters.
9. **Step 3: Click 'Try it out':** This enables the request form for that endpoint.
10. **Step 4: Upload a File:** Click 'Choose File' next to the file parameter and select an image (.jpg, .png, etc.), PDF (.pdf), or DOCX (.docx) from your computer. The file must be under 10MB.
11. **Step 5: Execute:** Click 'Execute'. The response appears below and shows: extracted metadata, risk score and level, SHA-256 file hash, and (for /scrub) a download URL for the scrubbed file.

Downloading Scrubbed Files

12. **Option A: Browser:** Copy the download_url from the /scrub response (e.g., /download/random_token_scrubbed.jpg). Prefix it with your server address: http://localhost:8000/download/random_token_scrubbed.jpg and open it in your browser. The file downloads automatically.
13. **Option B: curl:** curl -X GET "http://localhost:8000/download/random_token_scrubbed.jpg" -o scrubbed_file.jpg

Important: Scrubbed files are available for 5 minutes only. After expiry, the URL returns HTTP 404 and you must re-run /scrub.

Recommended Workflow

14. **Analyze first:** Upload your file to /analyze and review the metadata and risk score to understand what is exposed.
15. **Verify if needed:** Use /verify for a quick binary check, returns true/false for high-risk status without modifying the file.
16. **Scrub:** Upload to /scrub to receive a clean file with all metadata removed, and download it within 5 minutes.

3.5 API Response Examples

/analyze Response Structure

```
{
  "filename": "example.jpg",
  "secure_filename": "random_token.jpg",
  "original_file_hash": "sha256_hash_value",
  "metadata": {
    "file_hash": "...",
    "file_type": ".jpg",
    "camera_model": "iPhone 14 Pro",
    "camera_make": "Apple",
    "gps_coordinates": { "latitude": 32.0853, "longitude": 34.7818 }
  },
  "risk_assessment": {
    "risk_score": 50,
    "risk_level": "Medium",
    "score_breakdown": { "gps_coordinates": 40, "camera_model": 5, "camera_make": 5
}
  }
}
```

/scrub Response Structure

```
{
  "filename": "example.jpg",
  "original_file_hash": "sha256_original",
```

```
  "risk_assessment": { "risk_score": 50, "risk_level": "Medium", ... },
  "scrubbing": {
    "performed": true,
    "risk_level": "Medium",
    "scrubbed_file_hash": "sha256_scrubbed",
    "scrubbed_fields": ["GPSInfo", "DateTime", "Make", "Model"],
    "download_url": "/download/random_token_scrubbed.jpg"
  }
}
```

/verify Response Structure

```
{
  "filename": "document.pdf",
  "original_file_hash": "sha256_hash",
  "high_risk_metadata": true,
  "risk_assessment": {
    "risk_score": 80,
    "risk_level": "High",
    "score_breakdown": { "author_email": 30, "username": 20, "producer_version":
15, ... }
  }
}
```

3.6 Attack Flow Diagram

The following diagram illustrates the full attack flow in an unprotected system, and where MetaGuard intervenes:

```
User  →  Upload File (containing metadata)  →  Web Server
                                                  |
                              File stored/served WITHOUT metadata removal
                                                  |
         Public file / API returns metadata  ←────┘
                          |
  Attacker  →  Extract metadata (EXIF / PDF Info / DOCX Core Props)
                          |
             Information Disclosure:
             GPS coordinates  |  Author email  |  Username  |  Software version

MetaGuard intervention (after 'Web Server' step):
   → analyzer.py   : Extract & surface all metadata
   → risk_engine.py: Score and classify risk level
   → scrubber.py   : Remove all sensitive metadata fields
   → security_logging.py: Record events (masked, no sensitive data)
```

4. Defense Mechanisms & Prevention

4.1 Threat Model

MetaGuard is designed to prevent metadata-based privacy leakage. The threat model addresses the following attack vectors:

- **GPS Exposure:** GPS coordinates embedded in images can reveal sensitive locations: homes, workplaces, or secure facilities, enabling physical surveillance or targeted attacks.
- **Author Email Exposure:** Document authorship metadata can leak personal or corporate email addresses, enabling phishing or targeted social engineering.
- **Username Leakage:** Usernames and account identifiers in document metadata can expose internal user IDs or corporate directory structures.
- **Software Version Fingerprinting:** Producer/creator fields and version strings reveal exact software versions, enabling attackers to match against known CVE exploits.

The system assumes that files may be malicious or malformed, and that clients are untrusted and may attempt to bypass validation or access unauthorized download URLs.

4.2 Why Standard Defenses Fail

In many web systems, security focuses on validating file type and size, and preventing code execution (e.g., scanning for malware). Metadata is not treated as a separate information channel. The server accepts a file, checks the MIME type and extension, and saves or forwards it without processing EXIF fields, PDF Info dictionaries, or DOCX Core Properties.

As a result, even when a file is completely safe from the perspective of content and file type, sensitive information continues to be stored in its internal structure, and is exposed to anyone who receives the file or calls an API that returns metadata. Standard WAFs and AV scanning do not address this attack surface at all.

4.3 How to Prevent Metadata Leakage

Prevention requires active server-side metadata handling:

- **Server-Side Scrubbing:** Remove all metadata before storing or distributing files. MetaGuard's scrubber module removes EXIF from images (Pillow), clears PDF Info dictionary fields including Title, Author, Subject, Creator, Producer, CreationDate, and ModDate (PyPDF2), and clears DOCX Core Properties including title, author, subject, comments, last_modified_by, and revision (python-docx).
- **MIME Type Validation:** Use python-magic to validate MIME type from actual file content (not just extension). MetaGuard also cross-checks that the detected MIME type matches the file extension, a mismatch triggers rejection.
- **File Size Limits:** A 10MB maximum (MAX_FILE_SIZE in file_validation.py) reduces load and mitigates DoS attacks.

- **Secure Filenames:** Generated using secrets.token_urlsafe(32), eliminates path traversal, filename injection, and reuse attacks. The original filename is never used on disk.
- **No Path Exposure:** File system paths are never returned to clients in any API response.
- **TTL for Scrubbed Files:** Scrubbed files are available for download for only 300 seconds (5 minutes). After that, they are deleted and the download key is removed from SCRUBBED_FILES. This limits the exposure window and prevents URL reuse.
- **Structured Logging Without Sensitive Data:** The security_logging module records only approved fields. A blocklist (metadata, gps_coordinates, file_path, contents) prevents accidental leakage through log files.
- **Rate Limiting:** Per-IP sliding window rate limiting (20 requests per 60 seconds) on /analyze, /verify, /scrub, and /download mitigates automated scanning and API abuse.

4.4 Relevant Security Best Practices

- **Validate MIME types:** Use content-based detection (python-magic) in addition to extension checks.
- **Limit file size:** 10MB maximum reduces memory pressure and DoS risk.
- **Never trust user-uploaded files:** Use random filenames, restrictive permissions (0o600), and strict path validation.
- **Remove metadata before redistribution:** Apply scrubbing to all files regardless of apparent risk level.
- **Apply rate limiting:** Enforce per-IP limits before any processing; violations return HTTP 429 and are logged.
- **Structured logging without sensitive data:** Accept only named, approved fields in log events; reject or filter any field that could carry raw metadata.

4.5 Security Hardening Summary

- **Rate Limiting:** 20 requests/minute per IP on all endpoints, reduces abuse and brute-force attempts.
- **Download Expiration (TTL):** 5-minute window for scrubbed file downloads, after which files are deleted and URLs return 404.
- **Structured JSON Logging:** Events logged with masked IPs; no raw metadata, GPS, or file paths ever written to logs.
- **Input Validation:** Extension whitelist, MIME verification, file size enforcement, and strict filename checks prevent path traversal and unsupported file types.

5. Project Structure & Module Reference

The project uses a clean, modular architecture with clear separation of concerns across seven specialized modules:

Project Directory Layout:

```
metaguard/
├── main.py              ← FastAPI routes, TTL, download registry
├── analyzer.py          ← Metadata extraction (EXIF, PDF, DOCX) + SHA-256
├── scrubber.py          ← Metadata removal for images, PDFs, DOCX
├── risk_engine.py       ← Rule-based risk scoring engine
├── file_validation.py   ← MIME/extension/size validation, secure save
├── rate_limiter.py      ← In-memory per-IP rate limiting
├── security_logging.py  ← Structured JSON security logging
└── requirements.txt     ← Python dependencies
```

Module	Purpose
`main.py`	FastAPI entry point. Defines all routes: /, /health, /analyze, /verify, /scrub, /download/{filename}. Manages scrubbed file TTL registry, runs expired-file cleanup, applies rate limiting and security logging, and orchestrates all modules.
`analyzer.py`	Extracts metadata from all supported file types: EXIF (including GPS) from images via Pillow; PDF metadata (Author, Creator, Producer, dates) via PyPDF2; DOCX core properties (author, last_modified_by, etc.) via python-docx. Computes SHA-256 hashes.
`scrubber.py`	Removes metadata from files: strips EXIF from images (Pillow), clears PDF info dictionary fields (PyPDF2), and clears DOCX core properties (python-docx). Writes a new cleaned file and returns its path and the list of scrubbed field names.
`risk_engine.py`	Rule-based risk scoring engine. Assigns points for GPS, author email, username, software/producer/creator version strings, and camera make/model. Maps total score to Low (0-29), Medium (30-69), or High (70-100). Used by /analyze, /verify, and /scrub.
`file_validation.py`	Validates all uploads: allowed extension whitelist, MIME-type verification via python-magic, file size enforcement (max 10MB), and extension-MIME consistency check. Saves files with cryptographically random names and 0o600 permissions. Provides cleanup utility.
`rate_limiter.py`	In-memory per-IP sliding window rate limiter (20 requests / 60 seconds). Raises RateLimitError when exceeded. Used in main.py before processing every upload or download endpoint.
`security_logging.py`	Logs structured JSON security events (file_analyzed, file_scrubbed, file_downloaded, rate_limit_triggered) with masked IPs and without any sensitive fields, no metadata values, GPS data, file paths, or file contents.
`requirements.txt`	Python dependency list. Install with: pip install -r requirements.txt

6. Design Decisions & Limitations

6.1 Design Tradeoffs

In-Memory Rate Limiting vs. Redis

MetaGuard uses in-memory rate limiting (Python defaultdict and deques) rather than a Redis-backed solution. This eliminates external dependencies, reduces latency by removing network round-trips, and simplifies deployment. The tradeoff is that rate limits reset on application restart and are not synchronized across multiple server instances. This is acceptable for single-instance deployments.

Stateless Architecture (No Database)

No persistent database is used. All state is either in-memory or in temporary files. This approach prioritizes privacy (no persistent user file storage), simplifies deployment (no database migrations or connection management), and enables stateless horizontal scaling. The cost is that there is no persistent audit trail or cross-request state.

5-Minute TTL for Scrubbed Files

Scrubbed files expire and are deleted after 5 minutes. This limits the on-disk exposure window, prevents disk exhaustion from abandoned downloads, and minimizes the lifetime of any sensitive data remnants. Users are expected to download immediately, which is realistic for the primary use case of scrubbing before sharing.

Rule-Based Risk Engine vs. Machine Learning

A deterministic, rules-based scoring system was chosen over an ML model. This provides full transparency and auditability, critical for a security tool. The same inputs always produce the same output. No training data is required, eliminating privacy concerns around collecting metadata for training. Rules can be updated directly based on new threat intelligence. The tradeoff is less adaptability to entirely novel attack patterns, but metadata risk patterns are well-understood and stable enough that a rule-based approach is both effective and sufficient.

File-Based Temporary Storage vs. In-Memory Processing

Files are written to a temporary directory rather than processed entirely in-memory. This is because the underlying libraries (Pillow, PyPDF2, python-docx) expect file paths rather than byte streams. Keeping large files on disk also reduces memory pressure. The tradeoff is disk I/O overhead, which is acceptable for files up to 10MB.

6.2 Known Limitations

- **Rate Limiting Synchronization:** Rate limiting is per-instance and does not synchronize across multiple application nodes. Multi-instance deployments require an external solution such as Redis.
- **In-Memory State Persistence:** Rate limit counters and the scrubbed file registry reset on application restart. No persistent state is maintained between restarts.
- **Audit Log Retention:** Structured security logs are emitted to stdout but not retained long-term. External log aggregation infrastructure is required for persistence.
- **Scrubbed File Expiration:** Files expire after 5 minutes. If the user does not download in time, they must re-upload and re-scrub the original file.
- **File Size Constraint:** 10MB maximum. Larger files are rejected at the validation layer and cannot be processed.
- **Risk Detection Coverage:** The rule-based engine may not detect novel or unconventional metadata-based threats that do not match established patterns. Regular rule updates based on current threat intelligence are recommended.

7. References

Academic & Research Sources

[1] Kanich, C. et al., "Content Security Policy and Metadata Leakage" - Academic and industry context for metadata and information disclosure in web security.

[2] Chowdhury, N. K. et al., "Metadata in Digital Images: A Security Perspective" - Analysis of EXIF and GPS privacy risks in research and industry literature.

Official Documentation

[3] FastAPI Official Documentation. Available: https://fastapi.tiangolo.com/

[4] Python Pillow (PIL) - Pillow Documentation, Image Metadata and EXIF. Available: https://pillow.readthedocs.io/

[5] PyPDF2 Documentation - PDF Metadata. Available: https://pypdf2.readthedocs.io/

[6] python-docx Documentation - Document Core Properties. Available: https://python-docx.readthedocs.io/

OWASP References

[7] OWASP, "Information Disclosure" - OWASP Foundation. Available: https://owasp.org/www-community/vulnerabilities/Information_exposure

[8] OWASP, "Unrestricted File Upload" - OWASP Foundation. Available: https://owasp.org/www-community/vulnerabilities/Unrestricted_File_Upload

Relevant Links

MetaGuard API Interactive UI: http://localhost:8000/docs

Project's Github Repository: https://github.com/Azrieli-College-of-Engineering/final-project-nagham

Health Check Endpoint: http://localhost:8000/health

libmagicwin64 (Windows): https://github.com/pidydx/libmagicwin64

This report documents the MetaGuard project as implemented in the codebase.

Analytical explanations reference the architecture and modules as described and do not alter the code.